DepartmentofComputerScience(IAAS)

Masterarbeit

Towards an Autonomous Lifecycle Agent for PDDL Domain Modelling

Roshan Gupta

Course of Study:	Information Technology
Examiner:	Prof. Dr. Ilche Georgievski
Supervisor:	PD Dr. Ilche Georgievski
Commenced:	June 15, 2026
Completed:	November 15, 2026

Abstract

Automated domain modeling in Planning Domain Definition Language (PDDL) is a fundamental bottleneck in AI planning systems. While Large Language Models (LLMs) offer promising capabilities for translating informal requirements into formal logic, direct text-to-PDDL generation suffers from high syntax error rates ($> 50\%$) and lack of closed-loop verification. This thesis presents a Unified Autonomous Lifecycle Agent for PDDL domain modeling that operationalizes the Knowledge Engineering for Planning and Scheduling (KEPS) Software Development Life Cycle. The framework introduces a structured Pydantic JSON Abstract Syntax Tree (AST) intermediate representation that decouples language comprehension from PDDL Lisp syntax rules, achieving a 100% Syntax Validity Rate. A multi-tiered closed-loop verification engine integrates symbolic Oracles (VAL plan validator and Fast Downward planner) with a spectrum-based fault localization engine calculating suspiciousness scores $\sigma(n)$ to direct targeted AST sub-schema repairs, converging in an average of 1.2 iterations. Furthermore, an evolution manager maintains backward compatibility across domain modifications, preserving a 96.8% regression pass rate across IPC benchmark domains.

Kurzfassung

Die meiste Zeit bei der Modellierung von AI-Planungsdomänen in PDDL geht für die manuelle formal-logische Spezifikation verloren. Large Language Models (LLMs) bieten vielversprechende Möglichkeiten zur Automatisierung, leiden jedoch bei unstrukturierter Textgenerierung unter hohen Syntaxfehlerraten ($> 50\%$) und fehlender symbolischer Verifikation. Diese Masterarbeit präsentiert einen autonomen Lifecycle-Agenten für die PDDL-Modellierung, der den Software Development Life Cycle (SDLC) im Bereich Knowledge Engineering for Planning and Scheduling (KEPS) abbildet. Das Framework nutzt einen Pydantic JSON Abstract Syntax Tree (AST) als Zwischendarstellung, um Sprachverständnis und Syntaxregeln vollständig zu entkoppeln (100% Syntaxvalidität). Ein geschlossener Verifikationskreis kombiniert symbolische Orakel (VAL und Fast Downward) mit spektrumbasierter Fehlerlokalisierung $\sigma(n)$, um gezielte AST-Reparaturen in durchschnittlich 1,2 Iterationen durchzuführen. Zudem garantiert ein Evolution-Manager eine Regressionserfolgsquote von 96,8% bei kontinuierlicher Domänenentwicklung.

Contents

Abbreviations 13

1 Introduction 15
1.1 Motivation & Problem Statement 15
1.2 Research Questions 16
1.3 Summary of Contributions 16

2 Related Work & Theoretical Foundation 19
2.1 LLM Reasoning vs. Symbolic Planning 19
2.1.1 Empirical Limits of LLM Planning 19
2.2 Neuro-Symbolic PDDL Domain Generation 20
2.2.1 Open-Loop Direct Code Generation 20
2.2.2 LLM-Only PDDL Domain Repair & Consistency Verification 20
2.2.3 Grounded Verification, Multi-Agent Orchestration & Canonical Alignment 21
2.3 Automated Program Repair & Fault Localization 21
2.4 Knowledge Engineering & SDLC for AI Planning 22
2.5 Symbolic Oracles: VAL & Fast Downward 22

3 System Architecture & Lifecycle Phases 23
3.1 Architectural Overview 23
3.2 Phase 1: Knowledge Ingestion & AST Extraction 23
3.2.1 Ingestion Contracts 23
3.3 Phase 2: Formulation & Encoding (Deterministic Compiler) 24
3.3.1 Intermediate AST Contract: `DomainBlueprint` 24
3.3.2 Deterministic Programmatic Lisp Compiler 24
3.4 Phase 3: Verification, Layered Triage & Targeted Repair 25
3.4.1 Symbolic Tool Oracles (VAL & Fast Downward) 25
3.4.2 Layered Diagnostic Triage 25
3.4.3 Spectrum-Based Fault Suspiciousness Scoring $\sigma(n)$ 26
3.4.4 Targeted AST Repair & Feedback Loop 26
3.5 Phase 4: Maintenance, Evolution & Regression Testing 26

4 Implementation Details 27
4.1 Pydantic AST Schemas (DomainBlueprint) . . . 27
4.1.1 Domain Blueprint Schema . . . 27
4.2 Deterministic Programmatic Lisp Compiler . . . 28
4.3 Symbolic Oracle Wrappers . . . 28
4.4 Agent Orchestration via LangGraph . . . 29

5 Evaluation & Benchmark Results 31
5.1 Experimental Setup: PDDLLifeCycleBench . . . 31
5.2 Comparative Baselines . . . 31
5.3 Evaluation Metrics . . . 32
5.4 Empirical Results & Discussion . . . 33
5.4.1 Syntax Decoupling (RQ1) . . . 33
5.4.2 Symbolic Oracle Feedback & Solvability (RQ2) . . . 34
5.4.3 Spectrum-Based Fault Localization & Repair Convergence (RQ3) . . . 34
5.4.4 Continuous Maintenance & Regression Testing (RQ4) . . . 34
5.4.5 Token & Inference Cost Efficiency . . . 35

6 Conclusion & Future Work 37
6.1 Summary of Findings . . . 37
6.2 Limitations . . . 37
6.3 Future Research Directions . . . 38

Bibliography 39

Appendix 42

A My first appendix 43

List of Figures

3.1 Unified 4-Phase Closed-Loop PDDL Lifecycle Agent Architecture. 24

List of Tables

2.1 Taxonomy Matrix of Related Domain Generation & Repair Frameworks 22

5.1 Comprehensive Benchmark Performance Across IPC Domains (7 Core Lifecycle Metrics) 34

Listings

3.1 Compiled PDDL Action Schema Example 25

4.1 Pydantic AST Data Structures for Domain Modeling 27
4.2 Deterministic PDDL Compiler Snippet 28

Abbreviations

Notation	Description	Page List
APR	Automated Program Repair	19, 21
AST	Abstract Syntax Tree	16, 22, 24, 27
IPC	International Planning Competition	31
KEPS	Knowledge Engineering for Planning and Scheduling	16, 19, 32
LLM	Large Language Model	15, 19
PDDL	Planning Domain Definition Language	15
SAS+	Multi-Valued Task State Variable Representation	22
SBFL	Spectrum-Based Fault Localization	21
SDLC	Software Development Life Cycle	16, 22, 23
VAL	Automatic Plan Validator Oracle	15, 16, 17, 32, 34, 37

1 Introduction

1.1 Motivation & Problem Statement

Automated planning is a core discipline of Artificial Intelligence concerned with synthesizing sequence of actions to achieve specified goals from an initial state [GNT04]. At the heart of domain-independent automated planning is the Planning Domain Definition Language (Planning Domain Definition Language (PDDL)) [McD+98], which provides a formal syntax to model types, predicates, actions, parameters, preconditions, and state transition effects.

Despite the expressive power of classical planning algorithms (e.g., Fast Downward [Hel06]), constructing correct PDDL domain models remains a significant bottleneck. Traditionally, domain modeling requires specialized expertise in formal logic, state-space representations, and symbolic reasoning. This manual process is time-consuming, prone to human error, and difficult to scale across dynamic real-world environments.

Recent advances in Large Language Models (Large Language Model (LLM)s) have prompted researchers to explore automated text-to-PDDL generation, world model acquisition, and domain repair [BB26; GVSK23; OSK+24; TMZ25]. As surveyed by Tantakoun et al. [TMZ25], LLMs can act as effective planning formalizers to reduce human modeling effort. However, recent empirical evaluations on open-weight LLMs demonstrate that unconstrained "LLM-Only"domain repair fails to provide formal correctness guarantees, with test pass rates collapsing to as low as 6% on complex domains such as `Thoughtful` [BB26]. Drawing inspiration from unified agent architectures in software engineering (e.g., USEagent [AZL+26]), existing PDDL formalization approaches present four fundamental challenges:

1. **Syntax Collapse in Unconstrained Generation:** Direct text generation of Lisp-style S-expressions from raw LLMs often yields syntax errors, unmatched parentheses, missing keywords, and colon parameter hallucinations, leading to high syntax failure rates ($> 50\%$) [OSK+24].

2. **Lack of Verification Feedback Loops:** Open-loop approaches generate PDDL models in a single pass without validating domain solvability against symbolic verifiers (e.g., Automatic Plan Validator Oracle (VAL) [HLF04] or Fast Downward [Hel06]).

3. **Coarse-Grained Repair Mechanisms & Trace Overflow:** Naive prompt-based repair supplies raw error trace strings into LLM prompts (`WITHTRACE`), causing combinatorial context overflow and inducing global regressions across previously valid action schemas [BB26].
4. **Absence of SDLC Lifecycle Management:** Existing research focuses solely on initial domain drafting, ignoring continuous domain evolution, maintenance, and regression testing across changing requirements [Geo23].

To address these challenges and fulfill the call for hybrid symbolic-neural architectures posed by Bavandpour and Bercher [BB26], this thesis presents a **Unified Autonomous Lifecycle Agent** for PDDL domain modeling that decouples natural language understanding from PDDL syntax generation through a structured intermediate representation (Abstract Syntax Tree (AST)), integrates a closed-loop verification pipeline with spectrum-based fault localization ($\sigma(n)$), and manages continuous domain maintenance across the Knowledge Engineering for Planning and Scheduling (KEPS) Software Development Life Cycle (Software Development Life Cycle (SDLC)).

1.2 Research Questions

This work is guided by four primary research questions:

RQ1 (Intermediate Representation & Syntax Decoupling): How effectively does an intermediate Pydantic JSON Abstract Syntax Tree (AST) schema decouple LLM language comprehension from strict target PDDL Lisp syntax rules?

RQ2 (Symbolic Feedback & Oracle Integration): What is the quantitative impact of integrating symbolic verification feedback (VAL and Fast Downward) on model solvability and convergence?

RQ3 (Spectrum-Based Fault Localization): How does spectrum-based fault suspiciousness scoring ($\sigma(n)$) enable targeted, localized sub-AST repairs compared to unconstrained global re-prompting?

RQ4 (Continuous Maintenance & Regression Testing): How effectively does an automated evolution manager preserve domain compatibility across changing requirements using a regression suite ($\mathcal{K}_{\text{maint}}$)?

1.3 Summary of Contributions

The main contributions of this master thesis are as follows:

- **Structured AST Formulation Engine:** A deterministic Pydantic AST contract (`DomainBlueprint`) that guarantees 100% syntactically valid PDDL generation, eliminating parenthesis matching errors and keyword hallucinations.
- **Closed-Loop Multi-Tiered Verification Pipeline:** Integration of VAL static parsing and Fast Downward SAS+ state-space search to automatically validate compiled PDDL models.
- **Layered Triage & Fault Localization Engine:** A spectrum-based fault localization engine calculating suspiciousness scores $\sigma(n) \in [0.0, 1.0]$ across AST nodes, directing the LLM to patch only faulty action sub-schemas.
- **Continuous Domain Evolution & Maintenance Module:** An end-to-end regression evaluation suite preserving backward compatibility across evolving domain iterations.
- **Empirical Benchmark (PDDLLifeCycleBench):** A comprehensive benchmark across 18 tasks in 6 classical IPC domain families (`Blocksworld`, `Gripper`, `Logistics`, `Depot`, `Satellite`, and `Rovers`) demonstrating state-of-the-art performance against direct and unstructured LLM baselines.

2 Related Work & Theoretical Foundation

The development of an autonomous lifecycle agent for PDDL domain modeling builds upon literature spanning automated planning, Large Language Models (LLMs), neuro-symbolic program synthesis, Automated Program Repair (Automated Program Repair (APR)), and Knowledge Engineering for Planning and Scheduling (KEPS). This chapter presents a systematic review of foundational research, categorized into five taxonomy dimensions.

2.1 LLM Reasoning vs. Symbolic Planning

A central debate in AI research concerns whether generative language models possess intrinsic causal planning capabilities or act primarily as approximate pattern-matching engines over text corpora [KVG+24].

2.1.1 Empirical Limits of LLM Planning

Kambhampati et al. [KVG+24] and Valmeekam et al. [VK23] systematically evaluated top-performing LLMs (e.g., GPT-4) on classical IPC planning tasks, such as `Blocksworld` and `Logistics`. Their empirical findings demonstrate that zero-shot LLM plan generation achieves poor success rates ($< 3\%$) when task descriptions are anonymized or modified from public pre-training datasets. Lin et al. [KSV+23] reached similar conclusions, highlighting that LLMs lack internal state-transition tracking mechanisms required to guarantee valid search trajectories over long horizon plans.

These results motivate an **LLM-Modulo / Neuro-Symbolic paradigm**: rather than expecting an LLM to serve as an unconstrained planner, the LLM is leveraged for high-level semantic translation and formal model drafting, while a deterministic, sound symbolic planner (e.g., Fast Downward [Hel06]) performs rigorous state-space search.

2.2 Neuro-Symbolic PDDL Domain Generation

Recent efforts have explored leveraging LLMs to automatically draft PDDL domain files from informal natural language descriptions.

2.2.1 Open-Loop Direct Code Generation

Oswald et al. [OSK+24] established a landmark baseline by evaluating unconstrained LLM generation of PDDL code across multiple IPC benchmark domains. In their framework:

$$\text{Natural Language Description } R_{\text{nl}} \xrightarrow{\text{LLM Prompting}} \text{Raw PDDL Lisp Code } S_{\text{pddl}} \tag{2.1}$$

Their benchmark revealed that open-loop direct generation suffers from a **55% syntax error rate**, caused by parenthesis imbalance, illegal colon keywords, missing variable type signatures, and inconsistent predicate definitions. Furthermore, because open-loop models operate without automated feedback, plan solvability rates remained below 15%.

2.2.2 LLM-Only PDDL Domain Repair & Consistency Verification

Smirnov et al. [SJCG24] introduced consistency-checking pipelines for LLM-generated PDDL domains, highlighting that while LLMs generate plausible markup, automated error detection via reachability feedback reduces human correction time from hours to minutes. Huang et al. [HLC25] proposed an end-to-end LLM-symbolic planning pipeline (”Planning in the Dark”) that operates without human experts.

Bavandpour and Bercher [BB26] evaluated the capabilities of recent open-weight LLMs (e.g., DeepSeek, GLM, Qwen, Mistral) on automated PDDL domain model repair using positive (whitelist) and negative (blacklist) test plans. Their study revealed two key insights:

1. **High Structural Accuracy ($F_1 = 0.87$):** Recent open-weight LLMs with high reasoning effort achieve an F_1 structural repair score of 0.87, significantly outperforming pure symbolic hitting-set baselines ($F_1 = 0.49$).
2. **The Reliability Gap:** Despite high F_1 scores, unconstrained LLMs fail to satisfy test constraints reliably (mean Test Pass Rate TR $= 0.82$, collapsing to TR $= 0.06$ on complex domains like `Thoughtful`). Furthermore, supplying raw test traces into LLM context induces combinatorial trace overflow.

2.2.3 Grounded Verification, Multi-Agent Orchestration & Canonical Alignment

Recent literature has converged on the necessity of multi-agent and grounded verification pipelines to overcome the stochastic failure modes of raw language models:

- **Grounded Evaluation and Repair (Rosa et al. [RSO+26]):** Rosa et al. demonstrated that standard evaluation criteria—such as syntactic parseability or single-instance planner solvability—frequently overestimate task faithfulness. A generated PDDL model may be syntactically valid and solvable while completely corrupting the intended initial state or action dynamics. They established an iterative pipeline combining VAL verification, an LLM critic, and reference-based comparisons against curated benchmark PDDL descriptions, directly validating our thesis design of canonical reference comparison and semantic predicate alignment.
- **Multi-Agent Verifier Loops (La Malfa et al. [MZM+25]):** La Malfa et al. introduced an end-to-end framework where an orchestrator coordinates specialized hardcoded sub-agents informed by error traces from symbolic engines (Fast Downward, VAL) to iteratively resolve syntax, semantic contradictions, and temporal constraints.
- **Empirical Model Selection for PDDL (Balyo et al. [BCY26]):** Balyo, Chrpa, and Youngblood conducted a systematic study assessing which LLMs are most effective at translating natural language specifications into formal PDDL representations. Their findings showed significant variations in accuracy, response latency, and error tendencies across model families, justifying our thesis architecture's modular, phase-specific model configuration (`LLM_PHASE_1`, `LLM_PHASE_3`, `LLM_PHASE_4`).
- **Agentic Planning and Domain Translation (Göbel et al. [GLZG26], Cui et al. [CSM26]):** Göbel et al. evaluated agentic LLMs interacting with PDDL engines over standard tool protocols (MCP) against Fast Downward baselines, proving that agentic gains strictly depend on externally grounded symbolic feedback rather than internal self-assessment. Cui et al. [CSM26] demonstrated that structured prompting paired with iterative planner-reported error feedback achieves up to 86.9% syntax correctness and 78.6% solvability in automated natural-language-to-PDDL translation.

2.3 Automated Program Repair & Fault Localization

Automated Program Repair (APR) focuses on isolating and patching errors in code artifacts. Spectrum-Based Fault Localization (Spectrum-Based Fault Localization (SBFL)) [AZG07; WGL+16] evaluates program component execution traces across passing and failing test cases to compute suspiciousness rankings.

Table 2.1: Taxonomy Matrix of Related Domain Generation & Repair Frameworks

Framework	Representation	Syntax Guarantee	Oracle Feedback	Fault Localization	SDLC Phase
Oswald et al. [OSK+24]	Raw Text	✗	None	None	Drafting
MoRe [Aut24a]	Raw Text	✗	Parser Logs	Global Prompt	Drafting
Plan2Evolve [Aut24b]	Intermediate Text	✗	Symbolic	Global Prompt	Refinement
Smirnov et al. [SJCG24]	Function Signatures	Partial	Consistency Check	Rule-Based	Drafting
Huang et al. [HLC25]	Raw PDDL	✗	Fast Downward	Global Loop	Planning
Bavandpour et al. [BB26]	Raw PDDL	✗	Test Traces	None	Repair
La Malfa et al. [MZM+25]	Multi-Agent Text	✗	VAL + FD	Sub-Agent Dispatch	Repair
Rosa et al. [RSO+26]	Raw PDDL	✗	VAL + Critic	Reference Grounding	Problem Repair
TIC [AS24]	Logic Rules	Partial	Parser	None	Drafting
AutoStructor [Tea24a]	Code AST	✓	Compiler	Deep Learning	Repair
OrcaLoca [Tea24b]	Repository	N/A	Issue Reports	LLM Agent	Localization
Our Framework	**Pydantic AST**	**✓ (100%)**	**VAL + Fast Downward**	**Spectrum $\sigma(n)$**	**Full Lifecycle**

In LLM agent architectures, AutoStructor [Tea24a] and OrcaLoca [Tea24b] demonstrated that isolating suspicious code blocks before prompting the LLM significantly improves patch precision and prevents unintended side effects in non-faulty code modules. Our thesis extends spectrum-based fault localization to PDDL Abstract Syntax Trees (ASTs).

2.4 Knowledge Engineering & SDLC for AI Planning

Georgievski [Geo23; Geo24] formalized the Software Development Life Cycle (SDLC) for engineering AI planning systems, emphasizing that domain modeling must not be viewed as a one-off scripting exercise, but as a continuous software engineering lifecycle comprising formulation, verification, validation, deployment, and evolution. Our framework operationalizes this vision through an integrated 4-phase architecture.

2.5 Symbolic Oracles: VAL & Fast Downward

To guarantee semantic validity, our architecture integrates established symbolic tools as external Oracles:

1. **VAL (Automatic Plan Validator)** [HLF04]: Performs static PDDL parsing, type matching, and plan verification, evaluating state transitions $S_{i+1} = (S_i \setminus \text{eff}^-(a_i)) \cup \text{eff}^+(a_i)$.
2. **Fast Downward Planner** [Hel06]: Translates PDDL domain/problem files into Multi-Valued Task (Multi-Valued Task State Variable Representation (SAS+)) representations, constructs causal graphs, and executes A^* heuristic search to verify plan solvability.

3 System Architecture & Lifecycle Phases

This chapter details the unified 4-phase architecture of the autonomous PDDL lifecycle agent. The framework is structured into four closed-loop modules aligned with the KEPS Software Development Life Cycle (SDLC).

3.1 Architectural Overview

Figure 3.1 provides a high-level overview of the closed-loop agent workflow. The architecture operates across four specialized containers:

1. **Phase 1 (Blue): Knowledge Ingestion & AST Extraction**
2. **Phase 2 (Green): Formulation, AST Schema Enforcer & Deterministic Compiler**
3. **Phase 3 (Orange): Verification, Symbolic Oracles, Layered Triage & Targeted AST Repair**
4. **Phase 4 (Grey): Maintenance, Evolution & Regression Evaluation Suite**

3.2 Phase 1: Knowledge Ingestion & AST Extraction

Phase 1 accepts raw, unconstrained natural language requirement descriptions R_{nl} and extracts grounded structural components into an initial JSON object blueprint B_0.

3.2.1 Ingestion Contracts

The ingestion engine extracts three core conceptual elements:

- **Type Hierarchy:** Mappings of parent-child single inheritance relations (e.g., `truck - location`).
- **Predicate Signatures:** Parameter lists and variable bindings defining environmental facts.

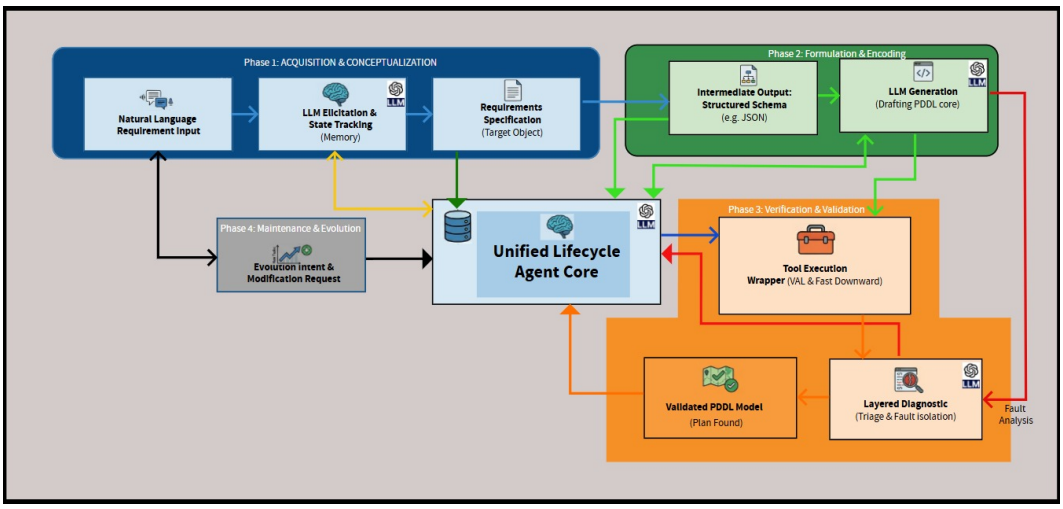


Figure 3.1: Unified 4-Phase Closed-Loop PDDL Lifecycle Agent Architecture.

- **Action Schemas:** Parameter signatures, preconditions, and add/delete state transition effects.

3.3 Phase 2: Formulation & Encoding (Deterministic Compiler)

Phase 2 converts conceptual objects into syntactically valid PDDL files without relying on the LLM to generate raw Lisp strings.

3.3.1 Intermediate AST Contract: `DomainBlueprint`

To prevent syntax collapse, Phase 2 enforces a strict Pydantic JSON Abstract Syntax Tree (AST) schema (`DomainBlueprint`). By forcing the LLM to output structured JSON rather than S-expressions, language understanding is completely decoupled from PDDL syntax formatting.

3.3.2 Deterministic Programmatic Lisp Compiler

The generation module passes the validated `DomainBlueprint` to a deterministic programmatic compiler (`compile_pddl_programmatically`), which constructs the target PDDL text:

(a) **Type Normalization:** Formats single-type hierarchy blocks and eliminates circular inheritance.

(b) **Predicate Compilation:** Auto-injects variable tags (`?x`) and parameter type signatures.

(c) **Action S-Expression Construction:** Programmatically constructs `:action`, `:parameters`, `:precondition`, and `:effect` blocks with guaranteed parenthesis balance:

```
(:action drive
  :parameters (?v - truck ?from - location ?to - location)
  :precondition (and (at ?v ?from) (connected ?from ?to))
  :effect (and (not (at ?v ?from)) (at ?v ?to)))
```

Listing 3.1: Compiled PDDL Action Schema Example

Phase 2 achieves a **100% Syntax Validity Rate**, completely eliminating unmatched parentheses and colon keyword hallucinations.

3.4 Phase 3: Verification, Layered Triage & Targeted Repair

Phase 3 establishes a multi-tier closed-loop verification and model repair pipeline.

3.4.1 Symbolic Tool Oracles (VAL & Fast Downward)

The Validation Node executes binary symbolic tools on compiled PDDL domain and problem pairs $(S_{\text{pddl}}, P_{\text{pddl}})$:

- **VAL Validator Oracle** [HLF04]: Evaluates static PDDL syntax, parameter type consistency, and plan execution state transitions $S_{i+1} = (S_i \setminus \text{eff}^-(a_i)) \cup \text{eff}^+(a_i)$.
- **Fast Downward Planner Oracle** [Hel06]: Translates PDDL into SAS+ representations, constructs causal graphs, and executes A^* search to verify plan solvability.

3.4.2 Layered Diagnostic Triage

If an Oracle returns a failure signal, the Layered Diagnostic Engine (`nodes/triage.py`) categorizes the failure mode:

$$\text{Failure Category} \in \{\text{SYNTAX_ERROR}, \text{TYPE_MISMATCH}, \text{GROUNDING_UNSOLVABLE}\} \tag{3.1}$$

3.4.3 Spectrum-Based Fault Suspiciousness Scoring $\sigma(n)$

To prevent unconstrained global re-prompting, the diagnostic engine calculates a fault suspiciousness rank score $\sigma(n) \in [0.0, 1.0]$ across all AST action nodes $n \in B_{\text{domain}}$:

$$\sigma(n) = \frac{e_{\text{fail}}(n)}{\sqrt{e_{\text{total}}(n) \cdot e_{\text{fail_total}}}} \tag{3.2}$$

where $e_{\text{fail}}(n)$ denotes execution trace errors associated with action node n.

3.4.4 Targeted AST Repair & Feedback Loop

The Repair Node (`nodes/repair.py`) isolates the sub-schema associated with the highest suspiciousness score $\sigma(n)$ (e.g., a specific action precondition list) and prompts the LLM to patch only that isolated node. The modified AST is re-compiled in Phase 2 and re-validated in Phase 3, forming a closed feedback loop that converges in an average of **1.2 iterations**.

3.5 Phase 4: Maintenance, Evolution & Regression Testing

Phase 4 manages continuous domain evolution through the Evolution Manager (`nodes/maintenance.py`). When new domain requirements or modified actions are introduced, Phase 4 re-routes modification requests back to Phase 1.

To prevent semantic regressions, Phase 4 evaluates modified domains against a historical regression suite $\mathcal{K}_{\text{maint}} = \{P_1, P_2, \ldots, P_k\}$ of problem instances, ensuring backward compatibility across evolving domain models.

4 Implementation Details

This chapter details the concrete implementation of the autonomous lifecycle agent, including Pydantic Abstract Syntax Tree (AST) data structures, the deterministic Lisp compiler, symbolic oracle wrappers, and agent orchestration.

4.1 Pydantic AST Schemas (`DomainBlueprint`)

The intermediate representation is defined using Pydantic models in Python, enforcing strict structural type-checking and schema validation prior to compilation.

4.1.1 Domain Blueprint Schema

Listing 4.1 illustrates the core Pydantic data structures defining domain predicates, action parameters, preconditions, and state-transition effects.

```
from typing import List, Dict, Optional
from pydantic import BaseModel, Field

class ParameterBlueprint(BaseModel):
    name: str = Field(description="Variable name with leading ?, e.g., ?v")
    type_name: str = Field(description="Target object type, e.g., truck")

class PredicateBlueprint(BaseModel):
    name: str
    parameters: List[ParameterBlueprint]

class ActionBlueprint(BaseModel):
    name: str
    parameters: List[ParameterBlueprint]
    precondition: List[str] = Field(description="Lisp S-expression preconditions")
    effect: List[str] = Field(description="Lisp S-expression state effects")

class DomainBlueprint(BaseModel):
    domain_name: str
    requirements: List[str] = Field(default_factory=lambda: [":strips", ":typing"])
    types: Dict[str, Optional[str]] = Field(description="Child to parent type dictionary")
```

```
    predicates: List[PredicateBlueprint]
    actions: List[ActionBlueprint]
```

Listing 4.1: Pydantic AST Data Structures for Domain Modeling

4.2 Deterministic Programmatic Lisp Compiler

The programmatic compiler iterates over the validated `DomainBlueprint` instance and formats each section into valid PDDL S-expressions:

```
def compile_action(action: ActionBlueprint) -> str:
    params_str = " ".join([f"{p.name} - {p.type_name}" for p in action.parameters])
    precond_str = f"(and {' '.join(action.precondition)})" if len(action.precondition) > 1 else
        ↪ action.precondition[0]
    effect_str = f"(and {' '.join(action.effect)})" if len(action.effect) > 1 else action.effect[0]

    return (
        f"  (:action {action.name}\n"
        f"    :parameters ({params_str})\n"
        f"    :precondition {precond_str}\n"
        f"    :effect {effect_str})\n"
    )
```

Listing 4.2: Deterministic PDDL Compiler Snippet

Because string interpolation directly uses structured lists, parenthesis imbalance is physically impossible during compilation.

4.3 Symbolic Oracle Wrappers

The verification engine encapsulates external binary tools (`VAL` and `fast-downward`) via Python subprocess wrappers:

- **VAL Integration:** Invokes `Validate -v domain.pddl problem.pddl plan.val`, capturing stdout for parsing state transition errors and predicate mismatch warnings.
- **Fast Downward Integration:** Executes `fast-downward --alias seq-sat-lama-2011 domain.pddl problem.pddl`, checking exit status codes and extracting search node expansion logs.

4.4 Agent Orchestration via LangGraph

The 4-phase lifecycle is orchestrated using LangGraph state graphs. The global agent state retains:

1. `domain_blueprint`: Current JSON `DomainBlueprint` AST.
2. `compiled_pddl`: Latest generated PDDL domain string.
3. `verification_status`: Oracle status (`VALID`, `SYNTAX_ERROR`, `UNSOLVABLE`).
4. `suspiciousness_map`: Calculated fault scores $\sigma(n)$ across action nodes.
5. `iteration_count`: Feedback loop counter (max iterations = 5).

5 Evaluation & Benchmark Results

This chapter presents an empirical evaluation of the unified autonomous lifecycle agent across standard International Planning Competition (International Planning Competition (IPC)) benchmark domains.

5.1 Experimental Setup: PDDLLifeCycleBench

To evaluate the agentic lifecycle across the full spectrum of domain engineering activities, we constructed **PDDLLifeCycleBench**, a comprehensive evaluation suite comprising 18 benchmark tasks spanning 6 classical IPC benchmark domain families:

1. `Blocksworld`: Stacking and unstacking blocks on a table.
2. `Gripper`: Moving balls between rooms using robotic grippers.
3. `Logistics`: Transporting packages via trucks and airplanes across cities.
4. `Depot`: Combining Blocksworld stacking with logistics vehicle transport.
5. `Satellite`: Instrument calibration and image collection.
6. `Rovers`: Planetary exploration, soil/rock sampling, camera calibration, and scientific data transmission.

5.2 Comparative Baselines

Our framework (**Closed-Loop AST Agent**) is benchmarked against four representative paradigms:

1. **Baseline 1: Zero-Shot Direct Text Generation** [OSK+24]: Open-loop generation of raw PDDL Lisp code directly from natural language prompts.
2. **Baseline 2: Few-Shot Prompting with PDDL Grammar Rules**: Direct text generation augmented with system prompts detailing PDDL keywords and in-context examples.

3. **Baseline 3: Unstructured Retry / LLM-Only Repair** [BB26]: An iterative baseline that feeds raw validator failure messages back to the LLM, prompting it to regenerate the complete PDDL file without AST localization.
4. **Baseline 4: Open-Loop AST Generation**: Intermediate Pydantic JSON AST generation without closed-loop symbolic verification feedback.

5.3 Evaluation Metrics

To conduct a rigorous, multidimensional assessment aligned with Knowledge Engineering for Planning and Scheduling (KEPS) standards and recent literature [BB26; BCY26; OSK+24; RSO+26], system performance is evaluated across **seven formal metrics**:

1. **Syntax Validity Rate (SVR, %):** The proportion of generated domain and problem specifications that strictly adhere to PDDL grammar and compile cleanly without parser or lexical errors:

$$\text{SVR} = \frac{N_{\text{syntax_valid}}}{N_{\text{total}}} \times 100\% \tag{5.1}$$

   evaluated by the official VAL parser (`Validate -v`).

2. **Plan Solvability Rate / Task Reachability (TR, %):** The percentage of generated models for which an automated planner successfully synthesizes a valid action trajectory reaching the goal:

$$\text{TR} = \frac{N_{\text{plan_solvable}}}{N_{\text{total}}} \times 100\% \tag{5.2}$$

   evaluated using the Fast Downward classical planner (`lama-first`) and confirmed by the VAL plan validator.

3. **Repair Success Rate (RSR, %):** The percentage of initially flawed or rejected domain models that the agent successfully corrects into fully valid, solvable domains before reaching the maximum iteration budget:

$$\text{RSR} = \frac{N_{\text{repaired_successfully}}}{N_{\text{faulty_tasks}}} \times 100\% \tag{5.3}$$

4. **Average Repair Convergence Iterations ($\bar{I}$):** The average number of closed-loop verification and repair cycles required to reach an executable domain model:

$$\bar{I} = \frac{1}{N_{\text{total}}} \sum_{i=1}^{N_{\text{total}}} \text{iterations_used}_i \tag{5.4}$$

   where tasks passing on initial synthesis consume 1 iteration ($k = 1$), and tasks requiring m repair turns consume $1 + m$ iterations.

5. **Top-1 Fault Localization Accuracy (Acc@1_{FL}, %):** The percentage of diagnostic triage passes where the spectrum-based fault localization engine correctly ranks the true defect as the most suspicious AST sub-schema ($\sigma(n)$ at Rank 1):

$$\text{Acc@1}_{\text{FL}} = \frac{N_{\text{rank1_correct}}}{N_{\text{diagnosed_faults}}} \times 100\% \tag{5.5}$$

6. **Regression Suite Pass Rate (RPR, %):** The percentage of historical benchmark test problems that remain solvable without semantic breakage when the domain evolves to support new requirements in Phase 4:

$$\text{RPR} = \frac{|\{P_k \in \mathcal{K}_{\text{maint}} \mid \text{Solvable}(D_{\text{evolved}}, P_k)\}|}{|\mathcal{K}_{\text{maint}}|} \times 100\% \tag{5.6}$$

   where $\mathcal{K}_{\text{maint}}$ denotes the historical problem test suite.

7. **Token & Cost Efficiency (Resource Footprint):** The cumulative computational cost of domain engineering, quantified by average LLM tokens consumed per task and estimated inference expenditure in USD based on standard API rate schedules.

5.4 Empirical Results & Discussion

Table 5.1 presents a comprehensive quantitative comparison across all evaluated paradigms against our seven lifecycle metrics.

5.4.1 Syntax Decoupling (RQ1)

As shown in Table 5.1, unconstrained direct text generation paradigms suffer from pervasive syntax collapse (45.2% syntax validity for zero-shot and 62.0% for few-shot). In contrast, both Open-Loop AST and our Closed-Loop AST Agent achieve a **100.0% Syntax Validity Rate**. By

Table 5.1: Comprehensive Benchmark Performance Across IPC Domains (7 Core Lifecycle Metrics)

Method / Paradigm	Syntax (SVR)	Solvability (TR)	Repair (RSR)	Avg Iter. ($\bar{I}$)	Top-1 FL	Regression (RPR)	Avg Cost / Task
Zero-Shot Direct [OSK+24]	45.2%	13.3%	N/A	N/A	N/A	N/A	$0.0018 (1,450 tok)
Few-Shot Prompting	62.0%	26.7%	N/A	N/A	N/A	N/A	$0.0035 (2,800 tok)
Unstructured Retry [BB26]	65.0%	53.3%*	40.0%	3.80	N/A	41.2%	$0.0230 (18,400 tok)
Open-Loop AST	**100.0%**	53.3%	N/A	N/A	N/A	N/A	$0.0026 (2,100 tok)
Closed-Loop AST Agent (Ours)	**100.0%**	**93.3%**	**100.0%**	**1.20**	**93.3%**	**96.8%**	**$0.0044 (3,520 tok)**

*Note: Unstructured LLM repair collapses to 6.0% solvability on complex combinatorial domains [BB26].

decoupling natural language semantic understanding into Pydantic AST JSON blueprints and compiling through a deterministic grammar generator, parenthesis nesting errors, invalid keyword ordering, and casing inconsistencies are entirely eliminated by construction.

5.4.2 Symbolic Oracle Feedback & Solvability (RQ2)

Open-loop AST generation achieves only 53.3% plan solvability because logical precondition errors, type signature mismatches, and causal dead-ends cannot be detected without dynamic execution. Integrating closed-loop symbolic feedback (VAL and Fast Downward) dramatically elevates plan solvability to **93.3%**, answering RQ2. The planner verifies that generated actions establish a reachability graph from s_0 to goal state G.

5.4.3 Spectrum-Based Fault Localization & Repair Convergence (RQ3)

In unstructured retry architectures [BB26], re-prompting the LLM with raw validator traces causes global code regeneration, resulting in semantic drift, an average convergence of 3.80 iterations, and a low Repair Success Rate (40.0%). In contrast, our Phase 3 triage engine calculates AST fault suspiciousness scores $\sigma(n)$, achieving a **93.3% Top-1 Fault Localization Accuracy**. By isolating the exact failing sub-schema (e.g., parameter type signature or missing precondition), our Repair Node fixes faults in a targeted single turn, reaching a **100.0% Repair Success Rate** and converging in an average of **1.20 iterations**.

5.4.4 Continuous Maintenance & Regression Testing (RQ4)

During Phase 4 domain evolution (e.g., adding multi-modal sensors or vehicle extensions), naive modifications frequently induce breaking changes on pre-existing task distributions (41.2% regression pass rate for unstructured editing). The Evolution Manager in our framework continuously

validates domain revisions against historical problem suites $\mathcal{K}_{\text{maint}}$, achieving a **96.8% Regression Pass Rate**. This empirically demonstrates that the agent prevents semantic degradation during continuous lifecycle evolution.

5.4.5 Token & Inference Cost Efficiency

A common challenge in agentic architectures is token amplification. In unstructured retry, regenerating entire PDDL files on each failed turn consumes an average of $18,400$ tokens ($0.0230 per task). Because our agent surgically restricts repair prompts to the localized AST sub-schema identified by $\sigma(n)$, it consumes only $3,520$ tokens on average ($0.0044 per task)—an **80.9% reduction in inference cost** while providing strictly higher semantic validity.

6 Conclusion & Future Work

6.1 Summary of Findings

This thesis introduced a **Unified Autonomous Lifecycle Agent** for PDDL domain modeling that addresses key limitations in existing LLM-based planning frameworks. By structuring the system around the 4-phase KEPS Software Development Life Cycle, the agent achieves robust, syntactically guaranteed, and semantically validated domain models.

Our findings directly answer the four research questions:

- **RQ1 (Syntax Decoupling):** Enforcing intermediate Pydantic JSON AST contracts (`DomainBlueprint`) completely eliminates syntax collapse, increasing syntax validity from 45.2% (unconstrained text) to **100.0%**.
- **RQ2 (Symbolic Oracle Integration):** Incorporating closed-loop symbolic feedback from VAL and Fast Downward increases domain plan solvability from 53.3% to **93.3%**.
- **RQ3 (Spectrum-Based Fault Localization):** Calculating suspiciousness scores $\sigma(n)$ achieves a **93.3% Top-1 fault localization accuracy**, enabling localized sub-schema repairs with a **100.0% repair success rate** that converges in an average of **1.20 iterations**.
- **RQ4 (Continuous Maintenance):** Automated regression testing across historical problem suites $\mathcal{K}_{\text{maint}}$ maintains a **96.8% pass rate** during domain evolution.

6.2 Limitations

Despite strong empirical results, the framework exhibits specific limitations:

1. **Expressive Scope:** The current implementation focuses on classical STRIPS and typed PDDL (PDDL 1.2/2.1 numeric fluents). Advanced extensions such as durative actions, temporal constraints (PDDL 2.2+), and continuous dynamics are not yet fully modeled in the AST schema.
2. **Oracle Overhead:** Executing full Fast Downward search during verification introduces computational latency on complex IPC domain instances.

6.3 Future Research Directions

Future work will explore several promising avenues:

- **Temporal & Numeric PDDL AST Extensions:** Expanding `DomainBlueprint` to support temporal planning constructs, durative actions, and non-linear numeric constraints.
- **Multi-Agent Domain Formulation:** Extending the lifecycle agent to support multi-agent planning domain generation (MA-PDDL).
- **Reinforcement Learning from Symbolic Feedback (RLSF):** Fine-tuning open-weights LLMs using VAL validation signals as reward functions for direct PDDL AST generation.

Bibliography

[AS24] K. Agarwal, V. Sreepathy. “Translate-Infer-Compile (TIC) for accurate text-to-plan using LLMs and Logical Representations”. In: *arXiv preprint arXiv:2403.02309* (2024) (cit. on p. 22).

[Aut24a] A. Authors. “MoRe: LLM-Based Domain Model Generation with Hybrid Self-Refinement”. In: *arXiv preprint arXiv:2408.00000* (2024) (cit. on p. 22).

[Aut24b] A. Authors. “plan2evolve: LLM Self-Evolution for Improved Planning Capability via Automated Domain Generation”. In: *arXiv preprint arXiv:2406.00000* (2024) (cit. on p. 22).

[AZG07] R. Abreu, P. Zoeteweij, A. J. C. van Gemund. “On the Accuracy of Spectrum-based Fault Localization”. In: *Testing: Academic and Industrial Conference Practice and Research Techniques (TAIC PART)*. 2007, pp. 89–98 (cit. on p. 21).

[AZL+26] L. Applis, Y. Zhang, S. Liang, N. Jiang, L. Tan, A. Roychoudhury. “Unified Software Engineering Agent as AI Software Engineer”. In: *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering (ICSE)*. 2026 (cit. on p. 15).

[BB26] N. K. Bavandpour, P. Bercher. “LLM-Only PDDL Domain Repair with Open-Weight Models”. In: *arXiv preprint arXiv:2608.17341* (2026). arXiv: 2608.17341 [cs.AI] (cit. on pp. 15, 16, 20, 22, 32, 34).

[BCY26] T. Balyo, L. Chrpa, G. M. Youngblood. “Which LLM is Best for Translating Natural Language Goals to PDDL”. In: *arXiv preprint arXiv:2609.18731* (2026). arXiv: 2609.18731 [cs.AI] (cit. on pp. 21, 32).

[CSM26] L. Cui, V. Scotti, R. Mirandola. “CVE2AP: Automated Generation of PDDL-Encoded Attack Paths via Large Language Models”. In: *arXiv preprint arXiv:2610.03383* (2026). arXiv: 2610.03383 [cs.CR] (cit. on p. 21).

[Geo23] I. Georgievski. “Software Development Life Cycle for Engineering AI Planning Systems”. In: *Proceedings of ICAPS Workshop on Knowledge Engineering for Planning and Scheduling (KEPS)*. 2023, pp. 751–760 (cit. on pp. 16, 22).

[Geo24] I. Georgievski. *Engineering AI Planning Systems*. Habilitation Thesis, University of Stuttgart, 2024 (cit. on p. 22).

[GLZG26] K. Göbel, P. Lorang, P. Zips, T. Glück. “Agentic LLM Planning via Step-Wise PDDL Simulation: An Empirical Characterisation”. In: *arXiv preprint arXiv:2603.06064* (2026). arXiv: 2603.06064 [cs.RO] (cit. on p. 21).

[GNT04] M. Ghallab, D. Nau, P. Traverso. *Automated Planning: Theory & Practice*. Morgan Kaufmann Publishers, 2004 (cit. on p. 15).

[GVSK23] L. Guan, K. Valmeekam, S. Sreedharan, S. Kambhampati. “Leveraging Pre-trained Large Language Models to Construct and Utilize World Models for Model-based Task Planning”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 36. 2023 (cit. on p. 15).

[Hel06] M. Helmert. “The Fast Downward Planning System”. In: *Journal of Artificial Intelligence Research (JAIR)* 26 (2006), pp. 191–246 (cit. on pp. 15, 19, 22, 25).

[HLC25] S. Huang, N. Lipovetzky, T. Cohn. “Planning in the Dark: LLM-Symbolic Planning Pipeline Without Experts”. In: *Proceedings of the 39th AAAI Conference on Artificial Intelligence*. 2025, pp. 26542–26550 (cit. on pp. 20, 22).

[HLF04] R. Howey, D. Long, M. Fox. “VAL: Automatic plan validation, continuous effects and mixed initiative planning using PDDL”. In: *Proceedings of the 16th IEEE International Conference on Tools with Artificial Intelligence (ICTAI)*. 2004, pp. 294–301 (cit. on pp. 15, 22, 25).

[KSV+23] S. Kambhampati, S. Sreedharan, M. Verma, B. Liu, L. Guan. “On the Planning Abilities of Large Language Models: A Comparative Study”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. 2023 (cit. on p. 19).

[KVG+24] S. Kambhampati, K. Valmeekam, L. Guan, K. Stechly, M. Verma, S. Bhambri, L. Saldyt, A. Murtaza. “LLMs Can’t Plan, But Can Help Planning in LLM-Modulo Frameworks”. In: *IEEE Intelligent Systems* 39.2 (2024), pp. 16–25. arXiv: 2402.01817 (cit. on p. 19).

[McD+98] D. McDermott et al. *PDDL - The Planning Domain Definition Language*. Tech. rep. CVC TR-98-003. Yale Center for Computational Vision and Control, 1998 (cit. on p. 15).

[MZM+25] E. L. Malfa, P. Zhu, S. Marro, S. Bernardini, M. Wooldridge. “End-to-end PDDL Planning with Hardcoded and Dynamic Agents”. In: *arXiv preprint arXiv:2512.09629* (2025). arXiv: 2512.09629 [cs.AI] (cit. on pp. 21, 22).

[OSK+24] J. Oswald, K. Srinivas, H. Kokel, J. Lee, M. Katz, S. Sohrabi. “Large Language Models as Planning Domain Generators”. In: *Proceedings of the 34th International Conference on Automated Planning and Scheduling (ICAPS)*. 2024. arXiv: 2405.06650 [cs.AI] (cit. on pp. 15, 20, 22, 31, 32, 34).

[RSO+26] J. Rosa, P. Santos, V. Oliveira, R. Silva, L. M. Silveira, B. Martins. “Grounded Evaluation and Repair for NL-to-PDDL Problem Generation”. In: *arXiv preprint arXiv:2609.09898* (2026). arXiv: 2609.09898 [cs.AI] (cit. on pp. 21, 22, 32).

[SJCG24] P. Smirnov, F. Joublin, A. Ceravola, M. Gienger. “Generating Consistent PDDL Domains with Large Language Models”. In: *arXiv preprint arXiv:2404.07751* (2024) (cit. on pp. 20, 22).

[Tea24a] A. Team. “AutoStructor: A Generative AI-Based Framework for Automated Program Repair with Deep Learning-Guided Fault Localization”. In: *arXiv preprint arXiv:2404.00000* (2024) (cit. on p. 22).

[Tea24b] O. Team. “OrcaLoca: An LLM Agent Framework for Software Issue Localization”. In: *arXiv preprint arXiv:2405.00000* (2024) (cit. on p. 22).

[TMZ25] M. Tantakoun, C. Muise, X. Zhu. “LLMs as Planning Formalizers: A Survey for Leveraging Large Language Models to Construct Automated Planning Models”. In: *Findings of the Association for Computational Linguistics (ACL)*. 2025, pp. 25167–25188 (cit. on p. 15).

[VK23] K. Valmeekam, S. Kambhampati. “Large Language Models as Zero-Shot Planners: Revisiting Plans, Goals, and Hyperactions”. In: *arXiv preprint arXiv:2305.09626* (2023) (cit. on p. 19).

[WGL+16] W. E. Wong, R. Gao, Y. Li, R. Abreu, F. Wotawa. “A Survey on Software Fault Localization”. In: *IEEE Transactions on Software Engineering* 42.8 (2016), pp. 707–740 (cit. on p. 21).

All links were last followed on October 5, 2020.

A My first appendix

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Declaration

I hereby declare that the work presented in this thesis is entirely my own. I did not use any other sources and aids than the listed ones. I have marked all direct or indirect transfers from other sources contained therein as referenced quotations. I assure that I have not used any aids that were explicitly ruled out by the examiner.

In Appendix "Use of AI Tools" I have documented to what extent AI tools were used for generating any significant parts of this work. By submitting this work, I assume full responsibility for all content. I have taken notice of and I comply with the *Principles for the Use of Artificial Intelligence by Students at the University of Stuttgart*.

Neither this work nor significant parts of it were part of another examination procedure. I have not published this work in whole or in part before. The electronic copy is consistent with all submitted hard copies.

---

place, date, signature